



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

Data Structures & Algorithms in Python

Lecture 02 – Linked List

Dr. Owen Noel Newton Fernando

College of Engineering

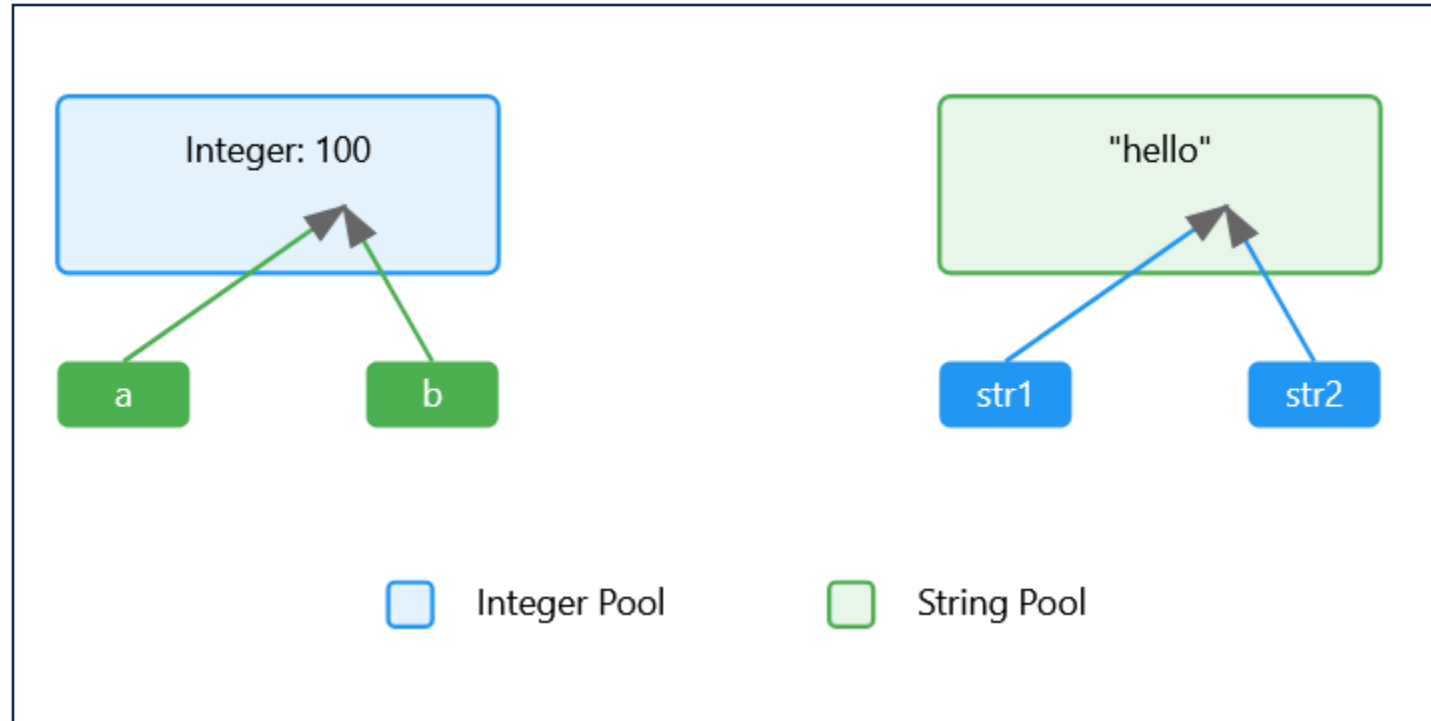
School of Computer Science and Engineering

Memory Pooling

- **Reduces Overhead:** Pre-allocating memory **reduces the need for frequent memory allocation and deallocation.**
- **Minimizes Fragmentation:** Reuses memory blocks efficiently, reducing gaps in memory.
- **Improves Performance:** **Faster memory allocation** due to pre-allocation and reuse.
- **Optimized for Small Objects:** Especially **beneficial for small integers, strings,** and frequently used types.

Memory Pooling: Example

```
1  # Memory Pooling Examples
2  # Integers (small numbers)
3  a = 100
4  b = 100
5  print(id(a))
6  print(id(b))
7  print("Same integers:")
8  print(a is b) # True
9
10 # Short strings
11 str1 = "hello"
12 str2 = "hello"
13 print("\nShort strings:")
14 print(str1 is str2) # True
```



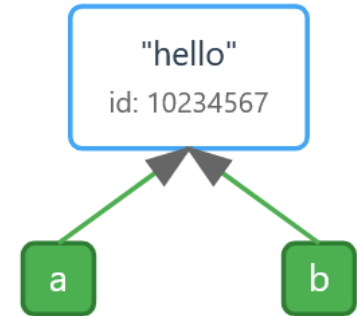
Memory Interning

- **Optimization Technique:** Python uses memory interning to enhance performance by **reusing objects instead of creating new ones** for frequently used immutable values.
- **Immutable Objects:** **Strings and small integers** are examples of immutable objects that are commonly interned automatically.
- **Memory Sharing:** Interned objects with the **same value share the same memory address**. This reduces memory usage and prevents unnecessary duplication.
- **Avoids Duplication:** By interning objects, Python avoids **creating multiple copies of the same immutable value**, which significantly reduces memory overhead.
- **Enhances Comparisons:** Interned objects improve performance during equality and identity comparisons since their **references can be directly compared instead of their values**.

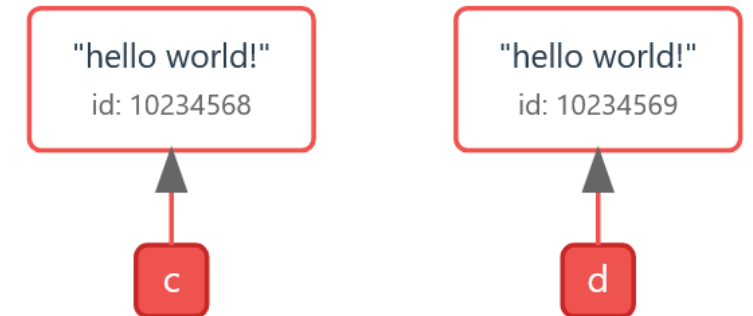
Memory Interning: : Example

```
1  # String literals share memory location
2  a = "hello"
3  b = "hello"
4
5  print(id(a)) # Memory location for 'a' (e.g., 140712834927872)
6  print(id(b)) # Memory location for 'b' (same as 'a')
7  print(a is b) # True - same memory location
8
9  # Longer strings - separate memory locations
10 c = "hello world!"
11 d = "hello world!"
12
13 print(id(c)) # Memory location for 'c' (e.g., 140712834928192)
14 print(id(d)) # Memory location for 'd' (different from 'c')
15 print(c is d) # False - different memory locations
```

String Literals Share Memory

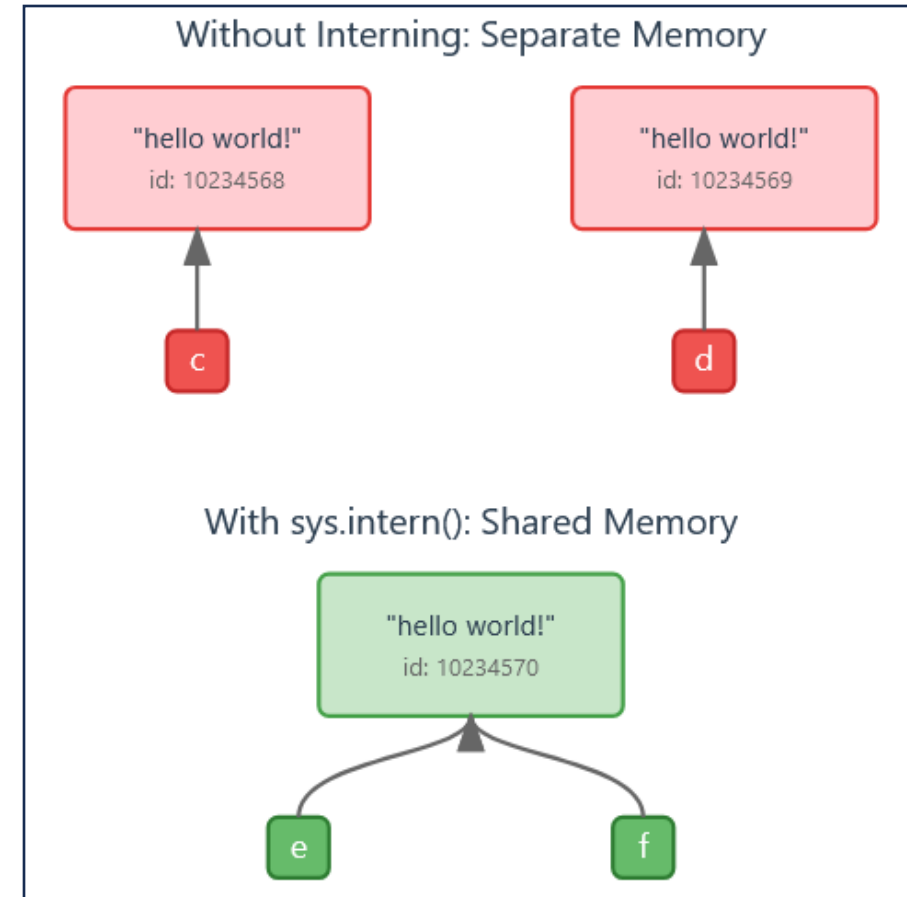


Longer Strings - Separate Memory



Memory Interning: : Manual Interning Example

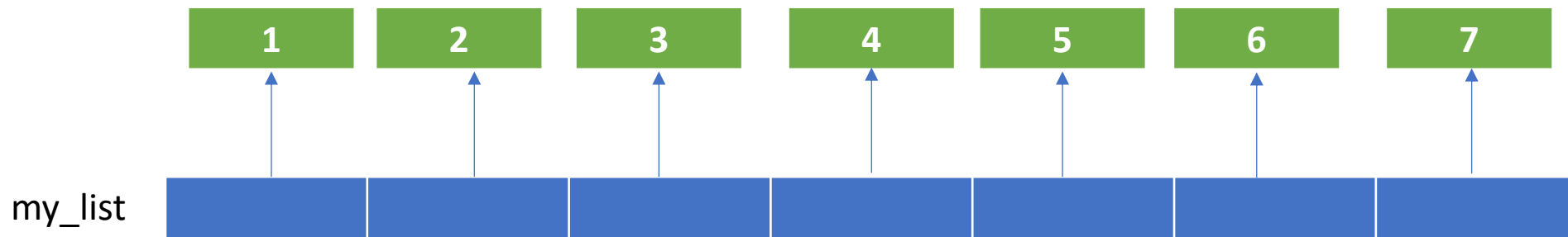
```
1  import sys
2  # Longer strings - separate memory locations
3  c = "hello world!"
4  d = "hello world!"
5
6  print(id(c)) # Memory location for 'c' (e.g., 140712834928192)
7  print(id(d)) # Memory location for 'd' (different from 'c')
8  print(c is d) # False - different memory locations
9
10 # Manual interning for large strings
11 e = sys.intern("hello world!")
12 f = sys.intern("hello world!")
13 print(id(e)) # Memory location for 'e'
14 print(id(f)) # Memory location for 'f'
15 print(e is f) # True - same memory location due to manual interning
```



Key Differences: Memory Pooling and Memory Interning		
Aspect	Memory Pooling	Memory Interning
Purpose	Improves efficiency by reusing preallocated memory blocks.	Reduces duplication by storing one shared copy of immutable objects.
Scope	Applies to small, frequently created objects (e.g., integers, short strings), by reusing preallocated memory blocks.	Applies specifically to immutable objects (e.g., strings, small integers), by storing one shared copy.
Automatic or Manual	Automatic for small objects (e.g., integers).	Automatic for short strings; manual interning possible for long strings.
Implementation	Reuses preallocated memory blocks	Reuses one shared copy of immutable values.
Example Use Case	Used for small integers and short strings.	Used for frequently repeated strings and small integers.

Referential Arrays

- A referential array typically refers to an array where each element is a reference (or pointer) to another object or data structure, rather than a direct value.
- In Python, everything is an object.
 - Its variables hold references to objects
 - The reference or pointer is the memory address where the data is located



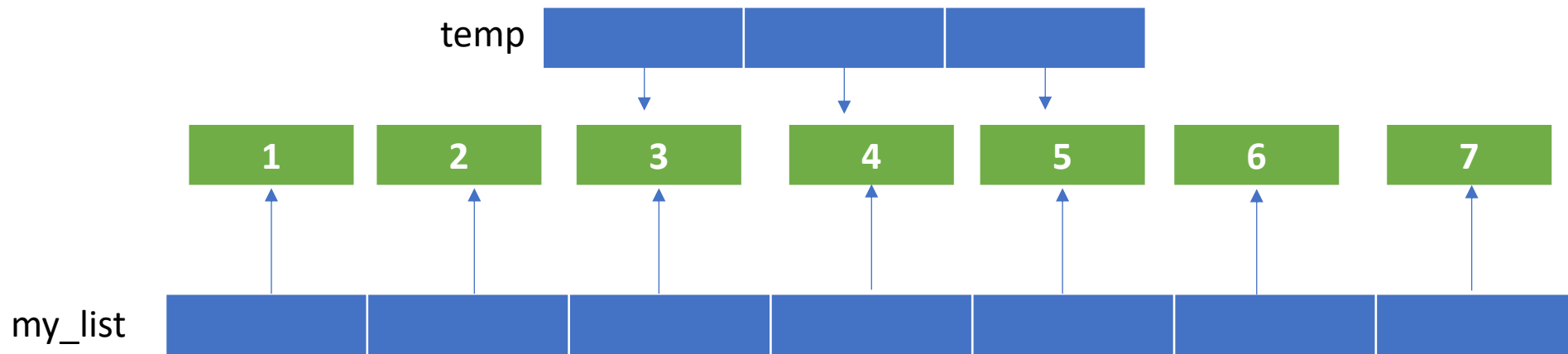
Referential Arrays

- create a list

```
>>my_list = [1, 2, 3, 4, 5, 6, 7]
```

- The fact in python is that each element is stored at different memory location

```
>>temp = my_list[2:5]
```



Referential Arrays

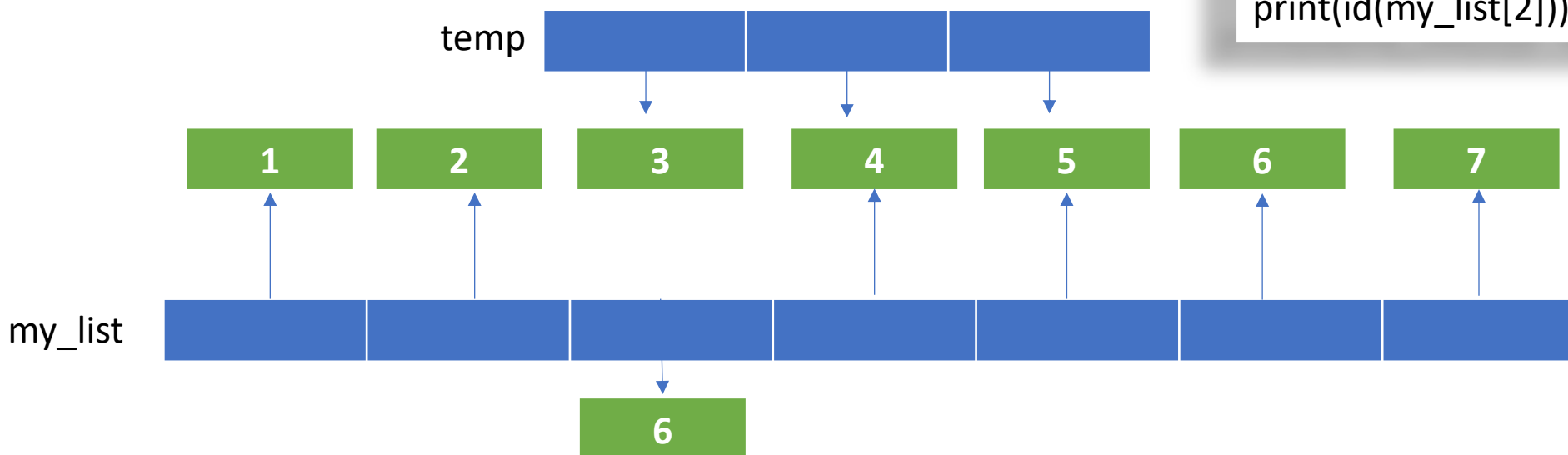
- create a list `>>my_list[2] = 6`
- `id()` return the memory address where the object is stored
- array module can more compactly represent an array
 - <https://docs.python.org/3/library/array.html>

```
my_list = [1, 2, 3, 4, 5, 6, 7]
```

```
print(id(my_list[2])) # memory address of object 3
```

```
my_list[2] = 6
```

```
print(id(my_list[2])) # memory address of object 6
```



Create A Customized Data Type Class

- Limited built-in data types in Python (any programming languages)
 - bool, int, float, complex (3+4j)..

Create A Customized Data Type Class

- Limited built-in data types in Python (any programming languages)
 - bool, int, float, complex (3+4j)..
- In practice, the data can be more complex
 - Student Information
 - Name – string
 - Age – int
 - CGPA – float (2 decimal places)
 - Etc.

```
class Student:  
    def __init__(self, name, age, cgpa):  
        self.name = name  
        self.age = age  
        self.cgpa = cgpa
```

Create A Customized Data Type Class

- Limited built-in data types in Python (any programming languages)
 - bool, int, float, complex (3+4j)..
- In practice, the data can be more complex
 - Student Information
 - Name – string
 - Age – int
 - CGPA – float (2 decimal places)
 - Etc.
- Benefits of Custom Data Types
 - **Encapsulation**: Bundle data and methods within a single unit.
 - **Reusability**: Create reusable code components.
 - **Abstraction**: Hide implementation details and expose only necessary operations.

```
class Student:  
    def __init__(self, name, age, cgpa):  
        self.name = name  
        self.age = age  
        self.cgpa = cgpa
```

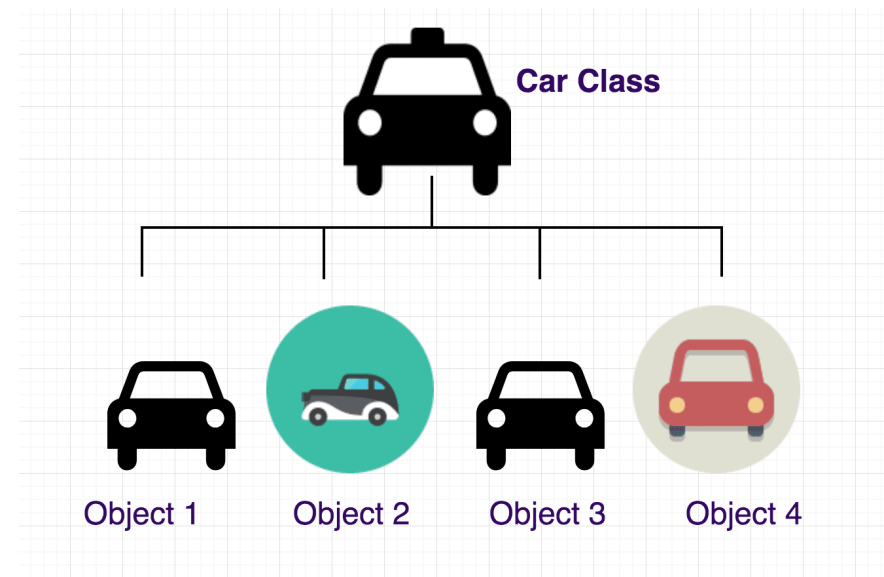
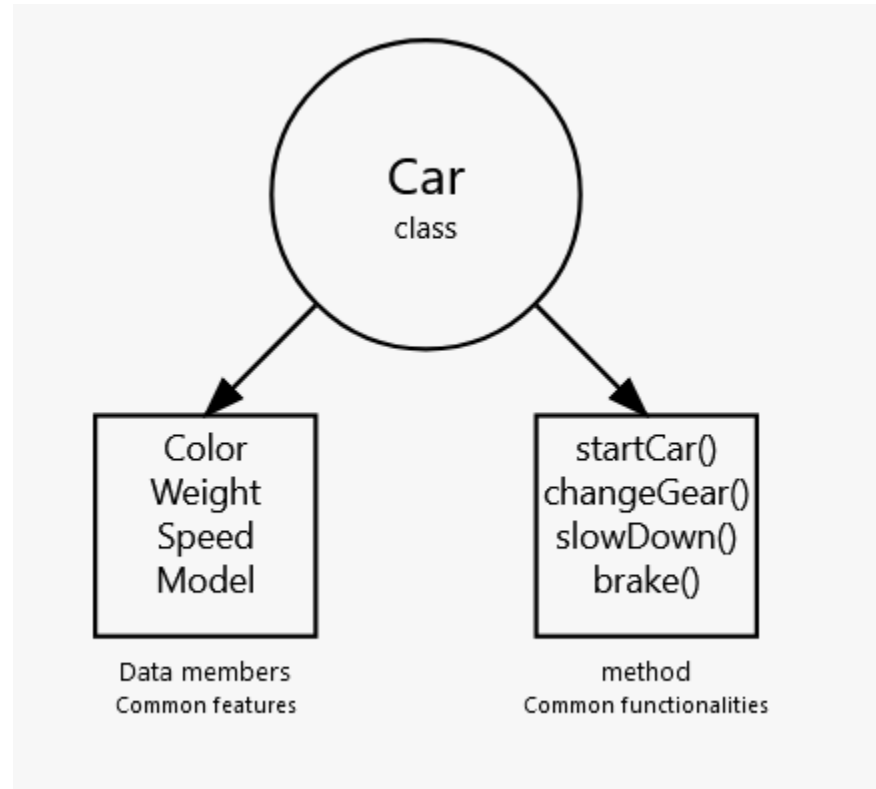
What is OOP?

A programming paradigm based on the concept of "**objects**" that encapsulate data and behavior.

OOPs is a way of organizing code that uses **objects** and **classes** to represent real-world entities and their behavior

In OOPs, **object** has **attributes** that have **specific data** and can perform certain actions using **methods**.

Object-Oriented Programming



Objects

- In Python, **everything**—numbers, strings, lists, dictionaries, etc.—is treated as an **object**.
- Objects represent **data** and provide mechanisms to interact with that data using **methods** and **attributes**.
- **Examples of Objects**
 - **Numbers**
 - 1234: An integer **(int) object**.
 - 3.14159: A floating-point number **(float) object**
 - **String:**
 - "Hello": **A string (str) object** that contains text.
 - **List:**
 - [1, 5, 7, 11, 13]: **A list (list) object** containing multiple integers.
 - **Dictionary:**
 - {"NTU": "Nanyang Technological University", "SG": "Singapore"}: **A dictionary (dict) object** mapping keys to values.

Objects

Every Object Has:

1. **A Type:**
 - Defines the **kind of data** the object represents (e.g., int, float, str, list, dict).
2. **An Internal Data Representation:**
 - **Primitive Representation:** Simple objects like int or float store data in a compact, raw form.
 - **Composite Representation:** Complex objects like list or dict combine multiple elements.
3. **Procedures for Interaction:**
 - **Objects expose methods** (functions associated with the object) for interaction.
 - Example: A string object ("Hello") has methods like .upper() or .lower().

An Object is an Instance of a Type

1. When you create an object, it is a **specific realization (instance)** of a broader **type (class)**.
 - Example:
 - 1234 is an **instance of the int type**.
 - "hello" is an **instance of the str type**.

Objects and Classes

Objects:

- Everything in Python is an **object**.
- Objects can be **created**, **manipulated**, and **destroyed** (e.g., using del or garbage collection).

Classes:

- Classes are **blueprints** for creating objects.
- A class defines **attributes (data)** and **methods (functions)**.
- Example: A Car class defines attributes (colour, speed) and methods (drive, brake). **Each car is an instance of the class.**

Creating and using your own types with classes

Creating a Class

- A class is a blueprint for creating objects.
- Steps Involved:
 - Define the class name (Student).
 - Specify the attributes (data) and methods (behaviour).

```
class Student:  
    institution_name = "NTU"  
  
    def __init__(self, name, age, cgpa):  
        self.name = name  
        self.age = age  
        self.cgpa = cgpa
```

- **institution_name** : A class attribute shared by all instances of the class.
- The **__init__** constructor method that runs automatically when a new Student object is created.
- **self** → Refers to the current object
- The Student class defines attributes like **name**, **age**, and **cgpa**.
- **name**, **age**, **cgpa** (parameters) values passed when creating an object to initialise its attributes.
- For instance, **self.name = name** assigns the value of **name** to the current object's **name** attribute.

Creating an instance of the class

Using a Class

Creating instances (objects) from a class and performing operations.

Example:

```
# Creating instances
student1 = Student("Newton", 20, 3.9)
student2 = Student("Fernando", 22, 3.7)

# Accessing attributes
print(student1.name)    # Output: Newton
print(student2.cgpa)    # Output: 3.7
```

- **student1** and **student2** are instances of the Student class.
- Attributes like **name**, **age**, and **cgpa** can be accessed directly.

When you create student1:

- When **student1** is created, the values "**Newton**", **20**, and **3.9** are passed to the **__init__** method of the **Student class**. The parameter **self** refers to the newly created **student1** object.
- Inside **__init__**, these values are assigned to the instance attributes **self.name**, **self.age**, and **self.cgpa**. As a result, **student1** stores its own values for name, age, and CGPA.
- The attribute **institution_name** is a class attribute. It is not stored separately in each object. Instead, it is shared by all instances of the Student class.

Accessing Attributes

After you have created **student1**, **student2** objects :

Creating instances

```
student1 = Student("Newton", 20, 3.9)
student2 = Student("Fernando", 22, 3.7)
```

Accessing instance attributes

```
print(student1.name)      # Output: Newton
print(student1.age)       # Output: 20
print(student1.cgpa)      # Output: 3.9
```

```
print(student2.name)      # Output: Fernando
print(student2.age)       # Output: 22
print(student2.cgpa)      # Output: 3.7
```

Accessing the shared class attribute

```
print(student1.institution_name) # Output: NTU
print(student2.institution_name) # Output: NTU
```

Accessing class attribute directly from the class

```
print(Student.institution_name)  # Output: NTU
```

Methods in classes

What is a method?

A method is a procedural attribute, like a **function that works only with this class**.

Python always passes the **object** as the first argument. The convention is to use **self** as the name of the first argument of all **methods**.

We will include methods to display the student's name, retrieve their age, and access their CGPA in our Student class.

Methods in classes

```
class Student:
    institution_name = "NTU"    # Class attribute

    def __init__(self, name, age, cgpa):
        self.name = name      # Instance attribute
        self.age = age        # Instance attribute
        self.cgpa = cgpa      # Instance attribute

    def get_name(self):        # Getter method for name
        return self.name

    def get_age(self):         # Getter method for age
        return self.age

    def get_cgpa(self):        # Getter method for CGPA
        return self.cgpa
```

Methods in classes

```
class Student:
    institution_name = "NTU"    # Class attribute

    def __init__(self, name, age, cgpa):
        self.__name = name    # Private instance attribute
        self.__age = age      # Private instance attribute
        self.__cgpa = cgpa    # Private instance attribute

    def get_name(self):        # Getter method for name
        return self.__name

    def get_age(self):         # Getter method for age
        return self.__age

    def get_cgpa(self):        # Getter method for CGPA
        return self.__cgpa
```


OOP Concept : Encapsulation

Encapsulation

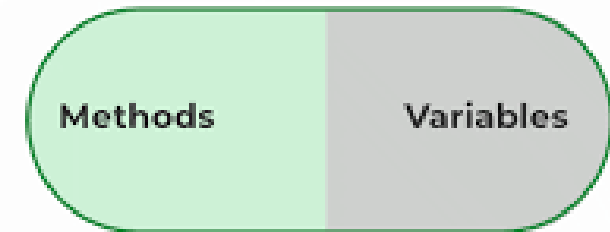
Encapsulation is the bundling of **data (attributes)** and **methods (functions)** within a class, restricting access to some components to control interactions.

Types of Encapsulation:

Public Members: Accessible from anywhere.

Protected Members: Accessible within the class and its subclasses.

Private Members: Accessible only within the class.



Class

OOP Concept : Encapsulation

Encapsulation

Python achieves encapsulation by:

Using a single underscore `_` attribute to indicate a **protected attribute** (convention, not enforced).

Using a double underscore `__` attribute to make the attribute **private** (name mangling to prevent direct access)

and

Providing controlled access via methods (getters and setters).

Public Attributes

- `self.name`, `self.age`, and `self.cgpa` are public attributes because they are not prefixed with an underscore (`_` or `__`).
- Public attributes can be accessed, modified, or used from anywhere, both inside and outside the class.

```
class Student:
    institution_name = "NTU" # Class attribute

    def __init__(self, name, age, cgpa):
        self.name = name # Instance attribute
        self.age = age # Instance attribute
        self.cgpa = cgpa # Instance attribute

    def get_name(self): # Getter method for name
        return self.name

    def get_age(self): # Getter method for age
        return self.age

    def get_cgpa(self): # Getter method for CGPA
        return self.cgpa

    def display_details(self): # Method to display student details
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        print(f"CGPA: {self.cgpa}")
        print(f"Institution: {Student.institution_name}")
```

OOP Concept : Encapsulation

Encapsulation

Python achieves encapsulation by:

Using a single underscore `_` attribute to indicate a **protected attribute** (convention, not enforced).

Using a double underscore `__` attribute to make the attribute **private** (name mangling to prevent direct access)

and

Providing controlled access via methods (getters and setters).

Encapsulation: Protected Attribute

- Attributes like `_name`, `_age`, and `_cgpa` are protected, following a convention of a single underscore prefix.
- Protected attributes are not enforced as private by Python but indicate to developers that they should not be accessed directly outside the class or its subclasses.

```
class Student:
    institution_name = "NTU"  # Class attribute

    def __init__(self, name, age, cgpa):
        self._name = name  # Protected instance attribute
        self._age = age    # Protected instance attribute
        self._cgpa = cgpa  # Protected instance attribute

    def get_name(self):      # Getter method for name
        return self._name

    def get_age(self):       # Getter method for age
        return self._age

    def get_cgpa(self):      # Getter method for CGPA
        return self._cgpa

    def display_details(self): # Method to display student details
        print(f"Name: {self.get_name()}")
        print(f"Age: {self.get_age()}")
        print(f"CGPA: {self.get_cgpa()}")
        print(f"Institution: {Student.institution_name}")
```

OOP Concept : Encapsulation

Encapsulation

Python achieves encapsulation by:

Using a single underscore `_` attribute to indicate a **protected attribute** (convention, not enforced).

Using a double underscore `__` attribute to make the attribute **private** (name mangling to prevent direct access)

and

Providing controlled access via methods (getters and setters).

Encapsulation: Private Attribute

If the attributes are private, they must be accessed using getter methods.

```
class Student:
    institution_name = "NTU"    # Class attribute

    def __init__(self, name, age, cgpa):
        self.__name = name    # Private instance attribute
        self.__age = age      # Private instance attribute
        self.__cgpa = cgpa    # Private instance attribute

    def get_name(self):        # Getter method for name
        return self.__name

    def get_age(self):         # Getter method for age
        return self.__age

    def get_cgpa(self):        # Getter method for CGPA
        return self.__cgpa

    def display_details(self): # Method to display all student details
        print(f"Name: {self.get_name()}")
        print(f"Age: {self.get_age()}")
        print(f"CGPA: {self.get_cgpa()}")
        print(f"Institution: {Student.institution_name}")
```

Create A Customized Data Type Class

- Limited built-in data types in Python (any programming languages)
 - bool, int, float, complex (3+4j)..
- In practice, the data can be more complex
 - Student Information
 - Name – string
 - Age – int
 - CGPA – float (2 decimal places)
 - Etc.
- Benefits of Custom Data Types
 - **Encapsulation**: Bundle data and methods within a single unit.
 - **Reusability**: Create reusable code components.
 - **Abstraction**: Hide implementation details and expose only necessary operations.

```
class Student:  
    def __init__(self, name, age, cgpa):  
        self.name = name  
        self.age = age  
        self.cgpa = cgpa
```


Example: Encapsulation

- Encapsulation **restricts direct access to a class's internal data** by hiding attributes and implementations. Interaction is managed through **public methods** to safeguard data integrity.
- Common Operations in **Encapsulation**
 - **Set**: Use methods to update attributes instead of direct modification.
 - **Print/Display/Get**: Retrieve the value of an attribute.
 - **Insertion**: Add a new object.
 - **Deletion**: Remove an object.
 - **Size**: Return the count of objects.

Create A Customized Data Type Class

```
1 class Student:
2     def __init__(self, name, age, cgpa):
3         self.__name = name
4         self.__age = age
5         self.__cgpa = cgpa
6
7     def get_name(self):
8         return self.__name
9
10    def get_age(self):
11        return self.__age
12
13    def get_cgpa(self):
14        return self.__cgpa
```

```
15 class StudentList:
16     def __init__(self):
17         self.__SL = []
18
19     def insert(self, name, age, cgpa):
20         self.__SL.append(Student(name, age, cgpa))
21
22     def delete(self):
23         self.__SL.pop()
24
25     def print(self):
26         for x in self.__SL:
27             print('name: ', x.get_name(), '\t age:
28                   ', x.get_age(), '\t cgpa: ', x.get_cgpa())
29
30 sl = StudentList()
31 sl.insert('Newton1', 10, 4.0)
32 sl.insert('Newton2', 30, 4.11)
33 sl.insert('Newton3', 20, 4.22)
34 sl.print()
35 print("After delete:")
36 sl.delete()
37 sl.print()
```

Private Access

Data Protection:

To ensure that critical data is not **accidentally modified by external code**.

Encapsulation:

To hide the internal representation of the data and expose only what is necessary through methods (getters and setters).

Immutability:

To create immutable objects where certain properties cannot be changed after the object is created.

In the example, we are not able to make any change once the Student data is created. No **setter methods** in the class Student and all attributes are private.

Heterogeneous Collection

List: This is the most commonly used data structure in Python.

- To handle heterogeneous collection, we define a new object class (eg. Student)
- `my_list = [1, 'hello', 3.14, True, [1, 2, 3]]`
 - You can do so but it is not recommended
 - **hard to read and manage**

Alternate solutions:

Dictionary: This is a collection of **key-value pairs**. It is unordered and mutable, and it allows for fast lookups, additions, and deletions of elements based on keys.

```
my_dict = {'integer': 1, 'string': 'hello', 'float': 3.14,  
           'boolean': True, 'list': [1, 2, 3] }
```

Any other solution?

Linked List

Linked List

- A **LinkedList** consists of **nodes** where each node has data and a pointer to the next node, ending with None.
- It's represented as sequential nodes connected by pointers in memory.

Key Operations:

- **Insert**: Add elements at start or **anywhere**
- **Delete**: Remove elements from **any position**
- **Search**: Find elements by **traversing through nodes**

Linked List

A LinkedList consists of nodes where each node has data and a pointer to the next node, ending with None. It's represented as sequential nodes connected by pointers in memory.

Key Operations:

- **Insert:** Add elements at start or anywhere
- **Delete:** Remove elements from any position
- **Search:** Find elements by traversing through nodes

Types

- **Singly Linked List** (one direction).
- **Doubly Linked List** (both directions).
- **Circular Linked List** (last node points to the first)

Linked List

A LinkedList consists of nodes where each node has data and a pointer to the next node, ending with None. It's represented as sequential nodes connected by pointers in memory.

Key Operations:

- **Insert:** Add elements at start or anywhere
- **Delete:** Remove elements from any position
- **Search:** Find elements by traversing through nodes

Types

- Singly Linked List (one direction).
- Doubly Linked List (both directions).
- Circular Linked List (last node points to the first)

Benefits:

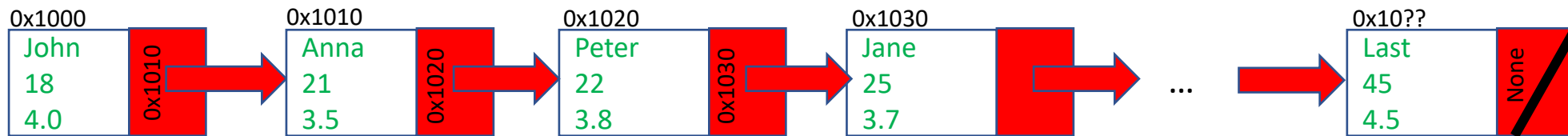
Dynamic memory allocation, efficient front insertions, flexible size management without wastage

Use Cases

Implementing stacks, queues, music playlists, browser history navigation

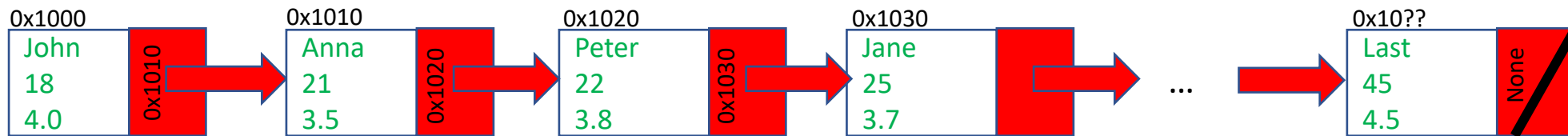
Singly Linked Lists

- A collection of nodes that collectively form a linear sequence.
- Each node stores
 - a reference to an **object** that is an element of the sequence
 - a **reference** to the **next node**



Singly Linked Lists

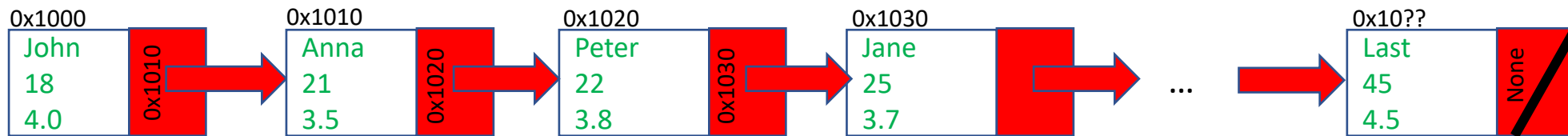
- A collection of nodes that collectively form a linear sequence.
- Each node stores
 - a reference to an object that is an element of the sequence
 - a reference to the next node



- In the Singly Linked List, there is only **ONE link** in each node.
 - Only the **previous** node can allocate the node

Singly Linked Lists

- A collection of nodes that collectively form a linear sequence.
- Each node stores
 - a reference to an object that is an element of the sequence
 - a reference to the next node



- In the Singly Linked List, there is only ONE link in each node.
 - Only the previous node can allocate the node
- Linked lists give more flexibility to manage the data.
- Constructing a linked list is more tedious than the list or dictionary.

Implementation of a linked list in Python

- A node class is required to store each **data** and the **link** to the next node

```
class Node:  
    def __init__(self, data, next):  
        self.data = data  
        self.next = next
```

Implementation of a linked list in Python

- A node class is required to store each data and the link to the next node

```
class Node:  
    def __init__(self, data, next):  
        self.data = data  
        self.next = next
```

- `def __init__(self, data, next):` This is the **constructor method** that **initializes** a **new Node object**:
 - `self` refers to the **instance being created** (Refers to current node instance)
 - `data` is the parameter that will store the **node's value**
 - `self.next`: Reference to the **next node** (default None, updated to link nodes).

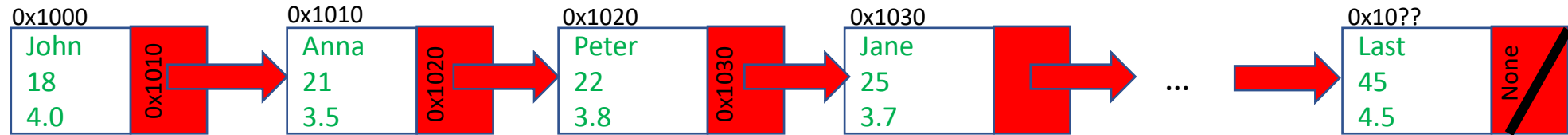
Implementation of a linked list in Python

- A node class is required to store each data and the link to the next node

```
class Node:  
    def __init__(self, data, next):  
        self.data = data  
        self.next = next
```

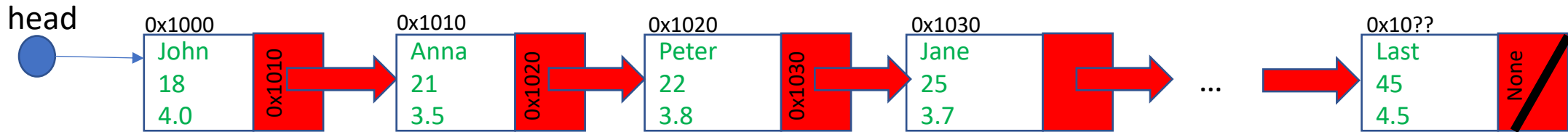
- `def __init__(self, data, next):` This is the constructor method that initializes a new Node object:
 - `self` refers to the instance being created (Refers to current node instance)
 - `data` is the parameter that will store the node's value
 - `self.next`: Reference to the next node (default None, updated to link nodes).
- Node Components:
 - `self.data = data` → Stores the value in node
 - `self.next = next` → Reference to the next node (empty at start)

Implementation of a linked list in Python



- A linked list class is used to create and manage a list of nodes.

Implementation of a linked list in Python



- A linked list class is used to create and manage a list of nodes.
- The **head reference** is essential to locate the first node in the list.
- Additional **reference** like a tail node can improve efficiency by pointing to the last node.
- The class supports operations such as insertion, deletion, and traversal to manage the list effectively.

```
class LinkedList:  
    def __init__(self):  
        self.head = None
```


Abstract Data Type (ADT)

ADT is a high-level abstraction for a data type, focusing on the operations that can be performed on it while hiding the implementation details.

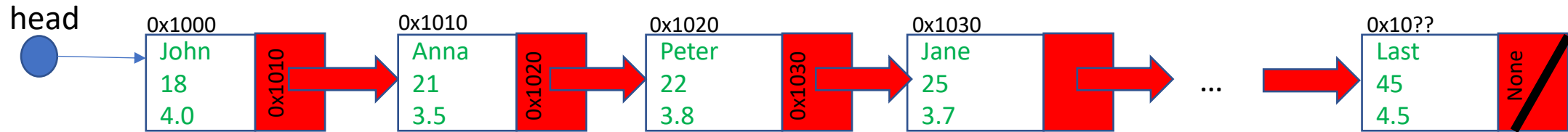
Specifies:

- **Data Type**: The kind of data the ADT can store.
- **Methods**: The operations that can be performed on the data (e.g., insertion, deletion, traversal).

Key Characteristics:

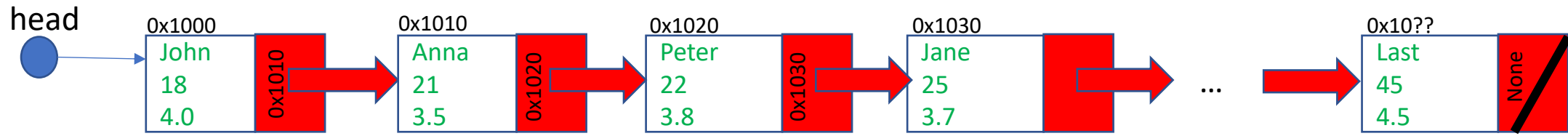
1. Focuses on **behaviour** rather than implementation.
2. **Encapsulates** the data and its operations.
3. **Hides internal workings** and implementation from the user.

Implementation of a linked list in Python



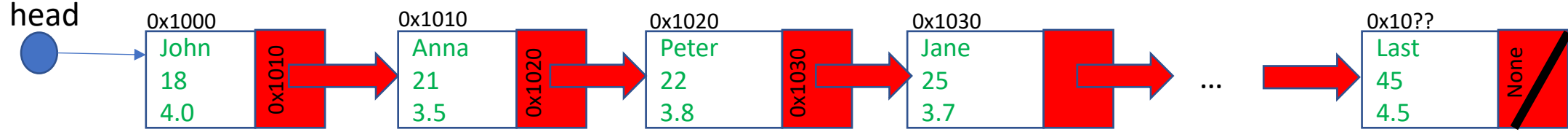
- To design the ADT of a simple Singly Linked List:
 - **Display** each element in the linked list
 - **Search** a node
 - **Add** a new node
 - **Remove** a node
 - **Size** of the linked list

Implementation of a linked list in Python



- **Display** each element in the linked list
 - Given the **head reference** of the linked list
 - Print all items in the linked list
 - From first node to the last node

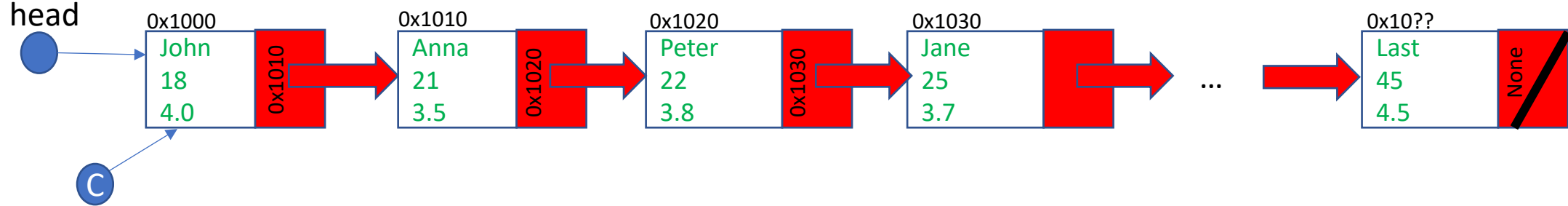
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

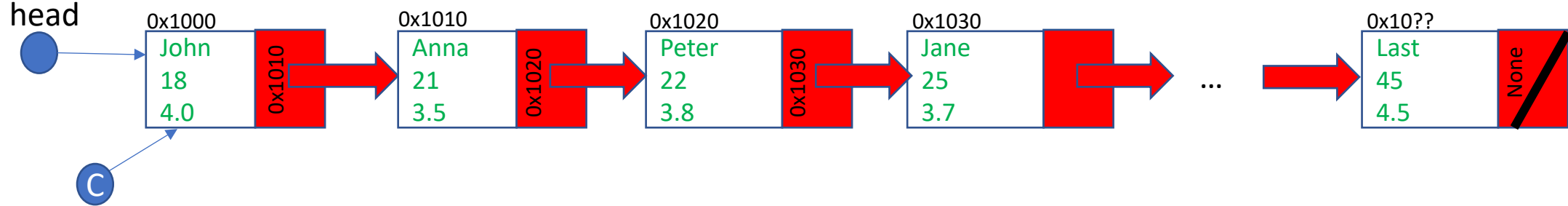
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

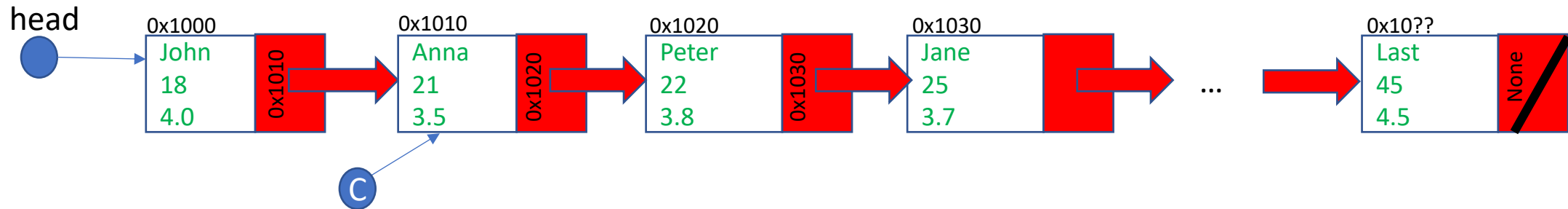
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

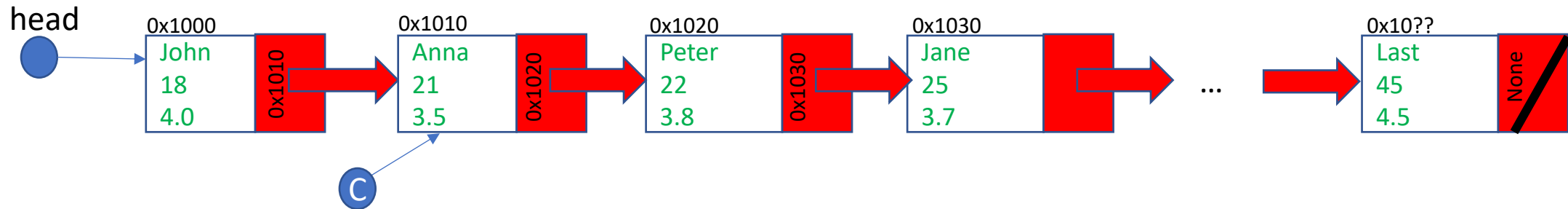
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

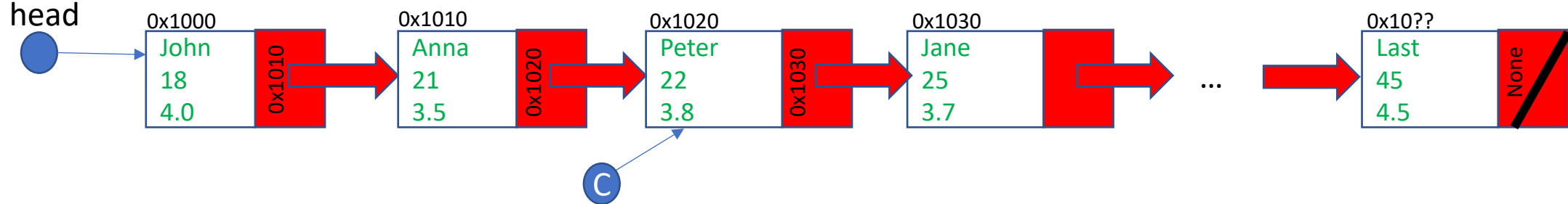
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

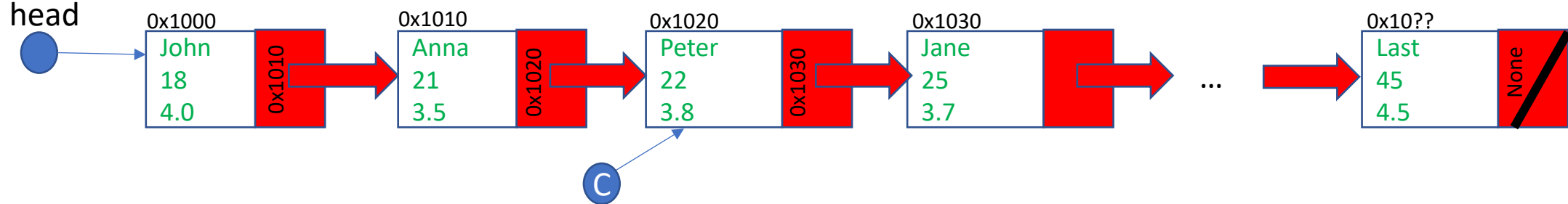

Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

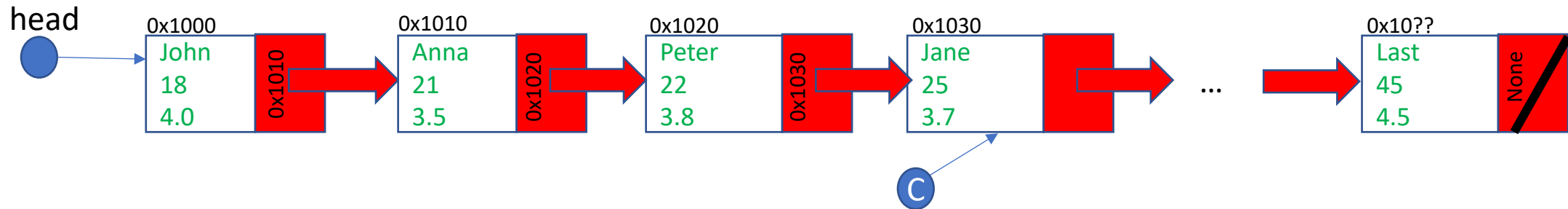
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

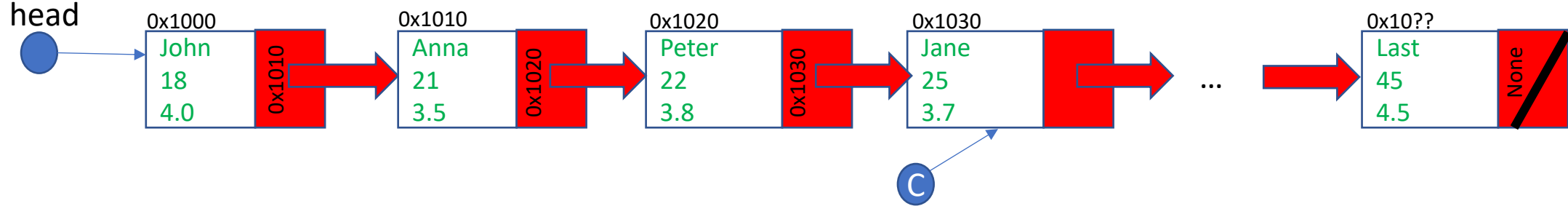
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

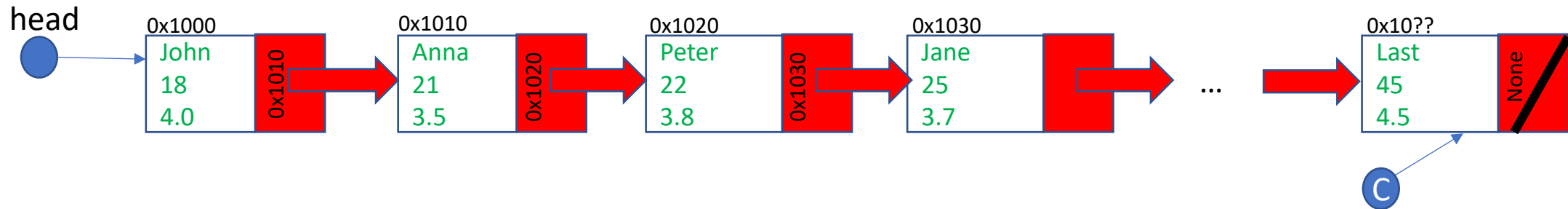
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

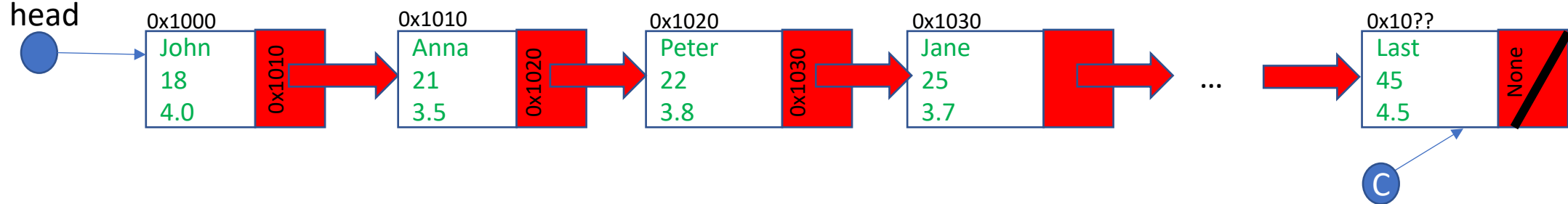
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

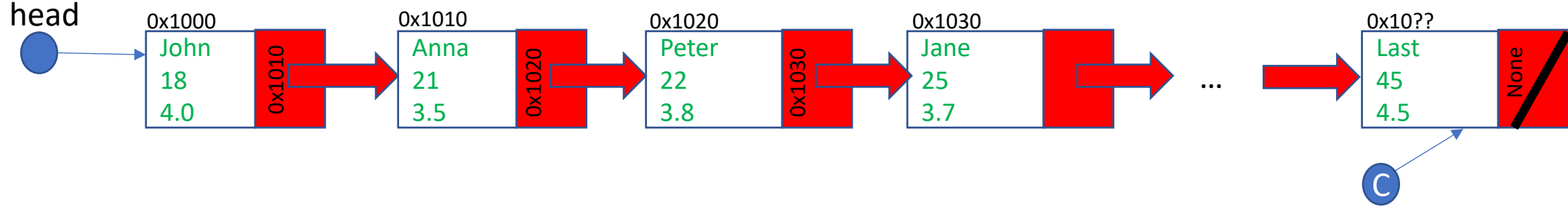
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```

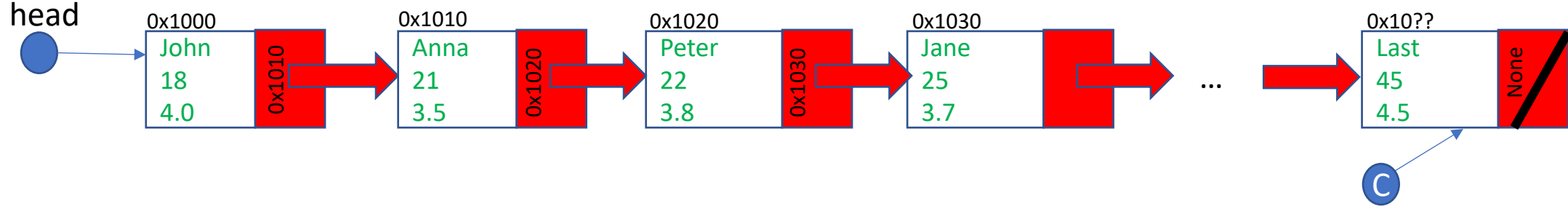
Display each element in the linked list



- Display each element in the linked list

```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next #None  
6     print("None")
```

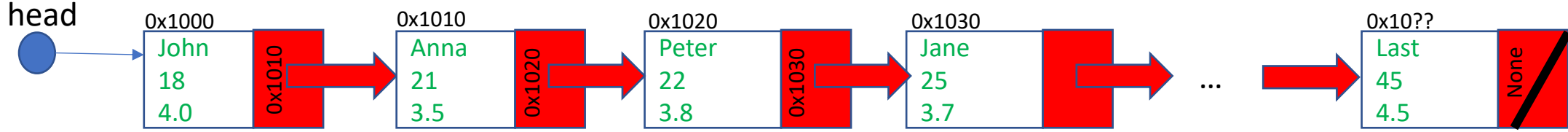
Display each element in the linked list



- Display each element in the linked list

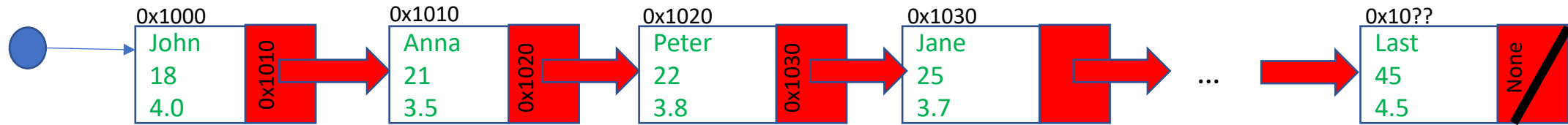
```
1 def display(self):  
2     current = self.head  
3     while current:  
4         print(current.data, end=" -> ")  
5         current = current.next  
6     print("None")
```


Search the node at index i



- Looking for the i th node in the list
 - Given the **head reference** of the linked list and **index i**
 - Return the pointer to the **i th** node
 - **None** will be return if index i is out of the range or the linked list is empty

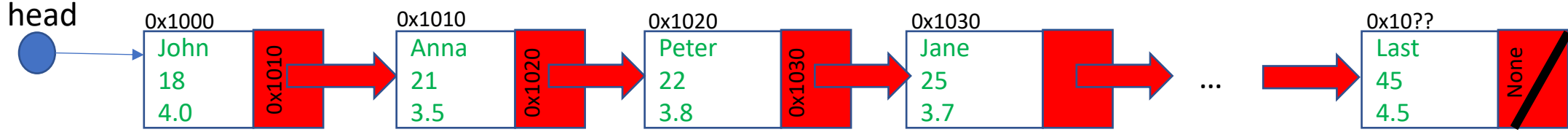
Search the node at index i



```
1 def findAt(self, index):  
2     current = self.head  
3     if not current:  
4         return None  
5     while index>0:  
6         current = current.next  
7         if not current:  
8             return None  
9         index-=1  
10    return current
```

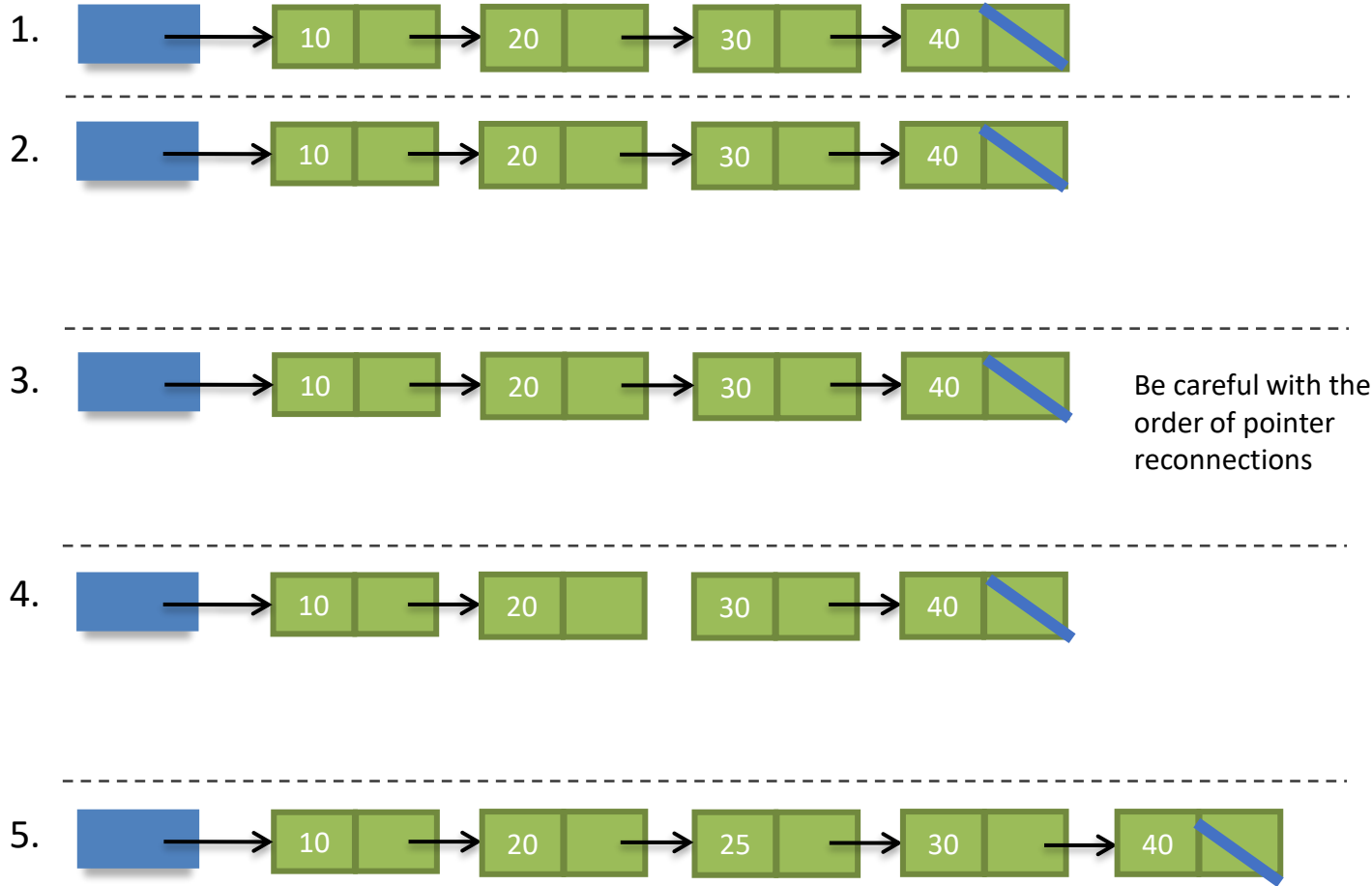
- Start from **head** (the reference to the first node).
- If the list is **empty** → **return None**.
- Traverse node by node, **decrementing index**.
- If you reach the end (None) **before finishing** → **return None**.
- When **index reaches 0** → return the reference to the current node.

Add a node

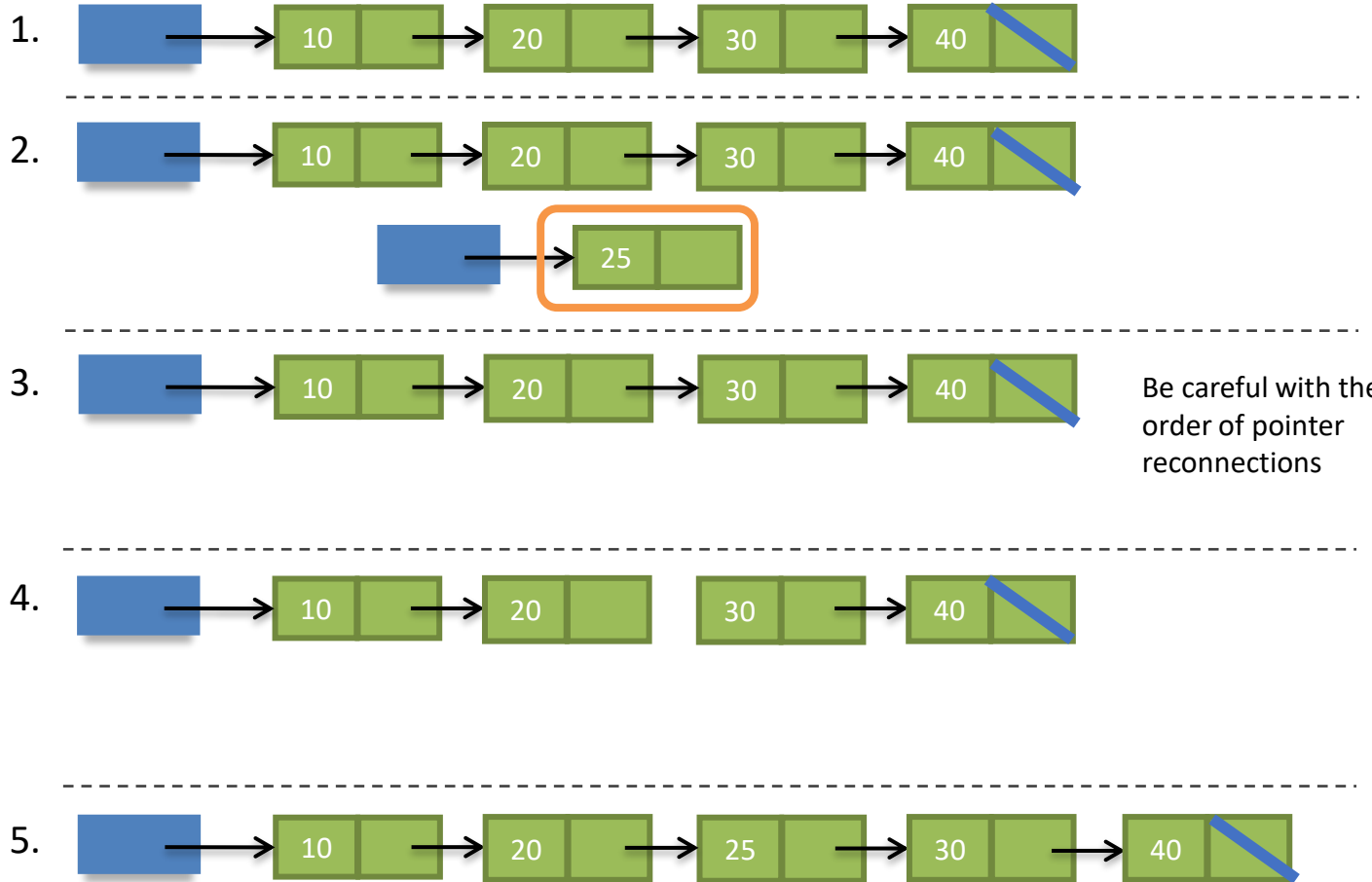


- Add a node in the linked list
- Given
 - the pointer to the **head reference** of a linked list
 - **index i** where the node to be inserted
 - the **item** for the node
- Return **SUCCESS** (1) or **FAILURE** (0)
 1. Create a node by the given item
 2. Insert the node at
 1. Front
 2. Middle
 3. Back

Insert a node in the middle

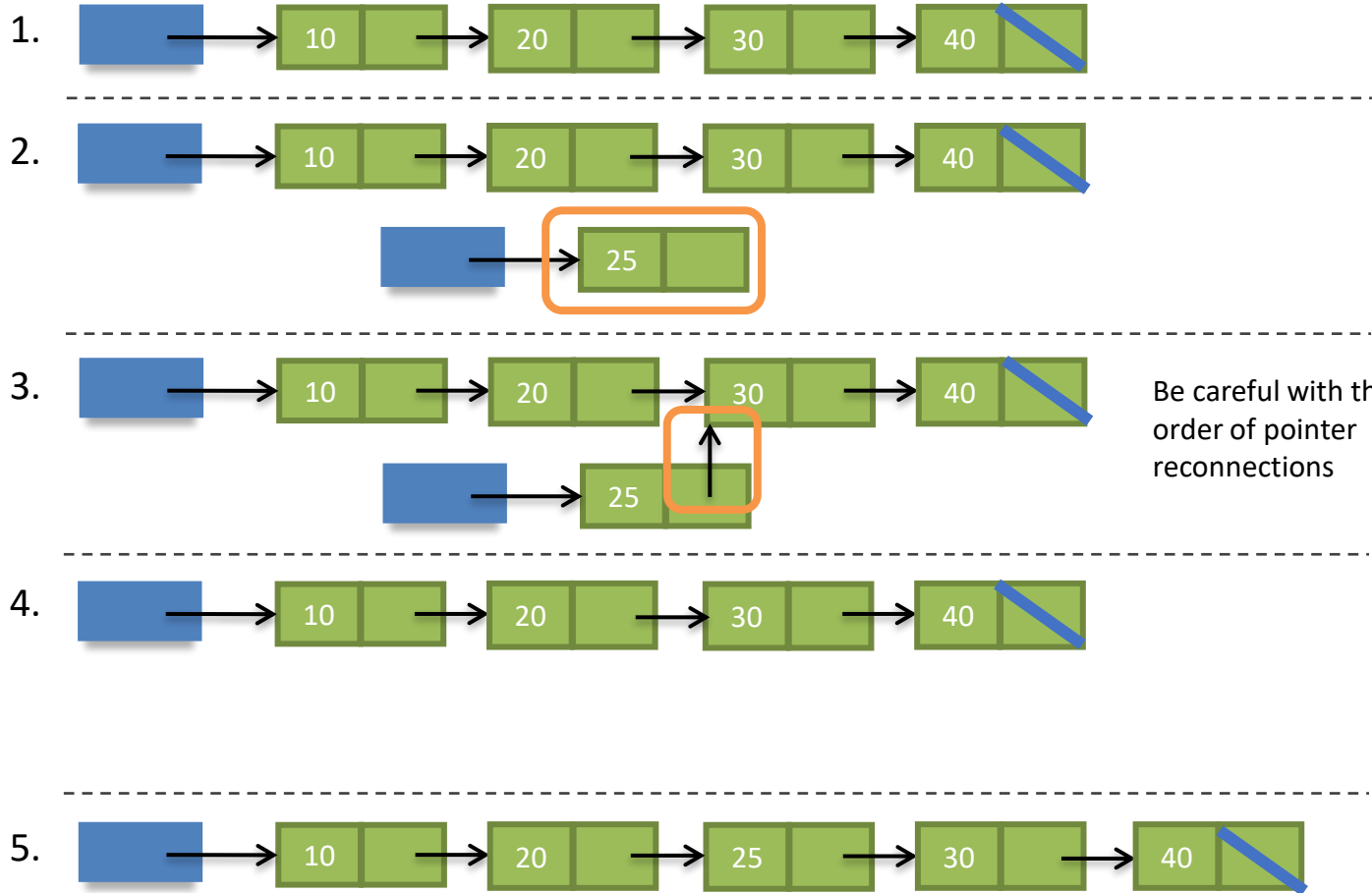


Insert a node in the middle



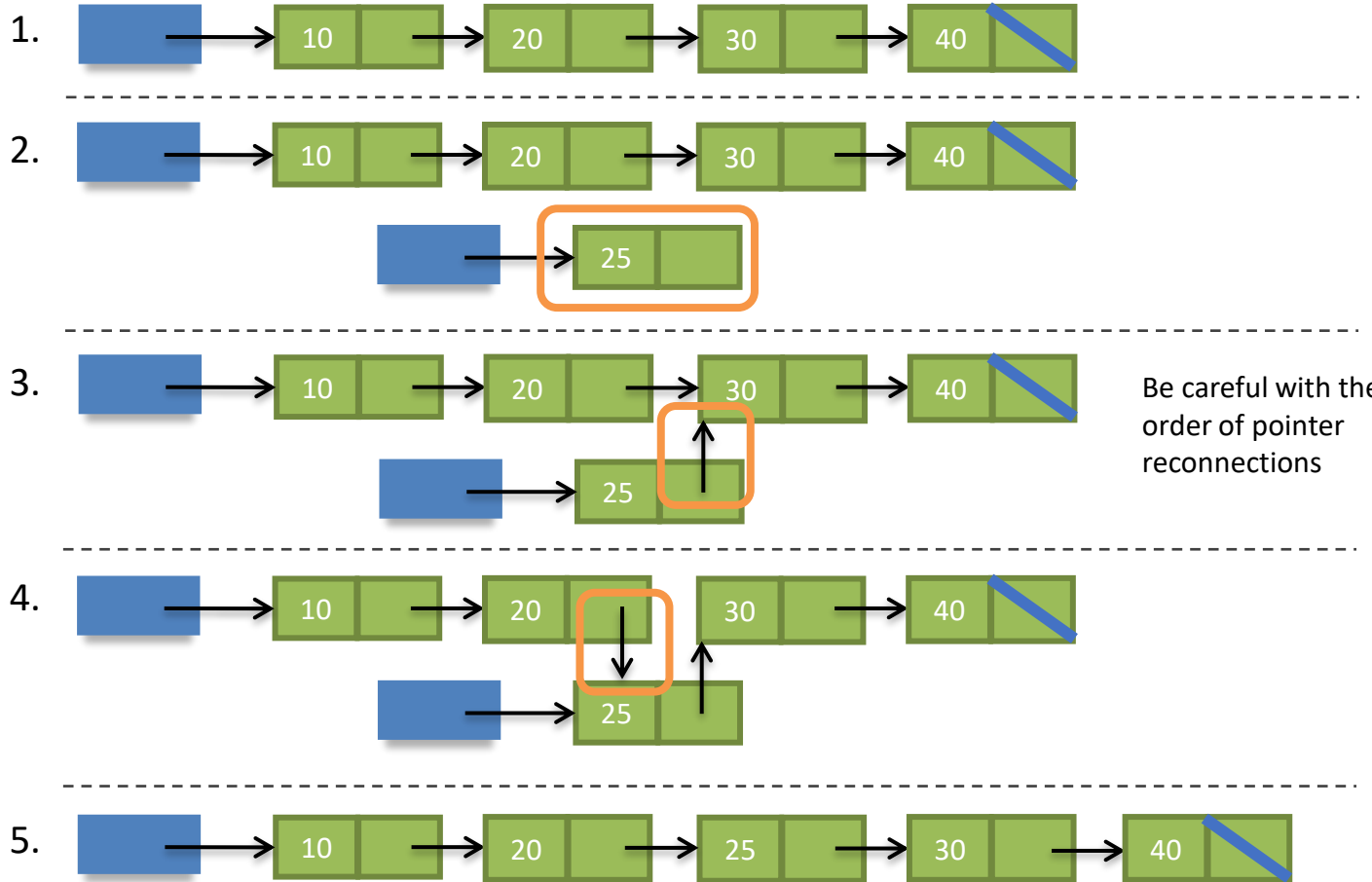
```
new_node = Node(data)
```

Insert a node in the middle



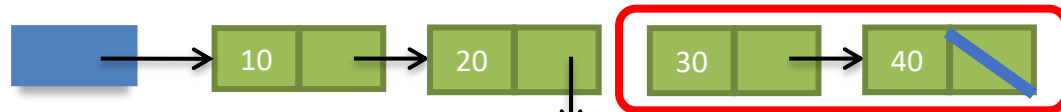
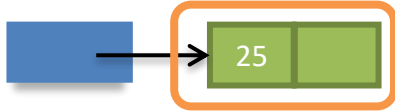
`new_node.next = pre.next`

Insert a node in the middle



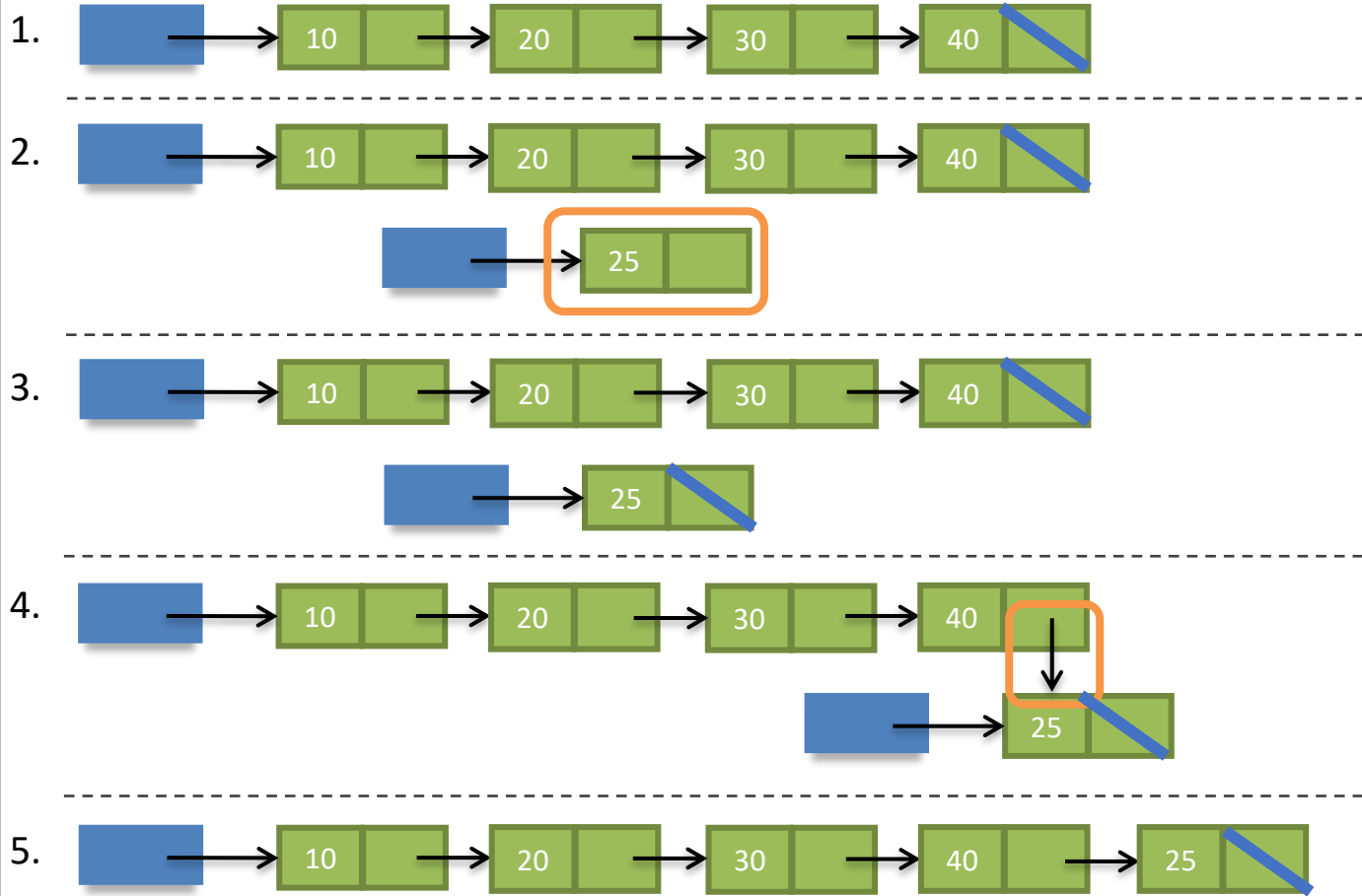
`pre.next = new_node`

Insert a node in the middle

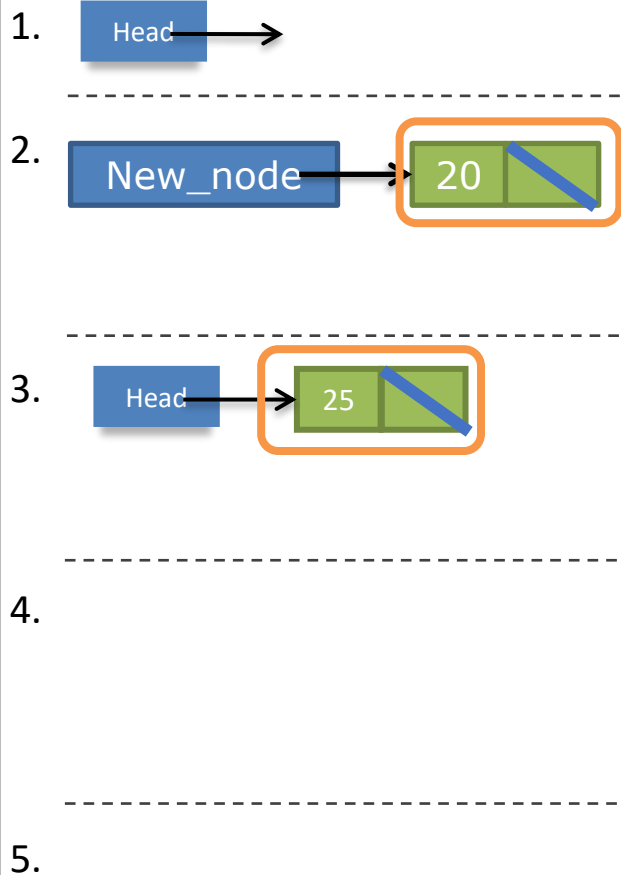


`pre.next = new_node`

Insert a node in the back

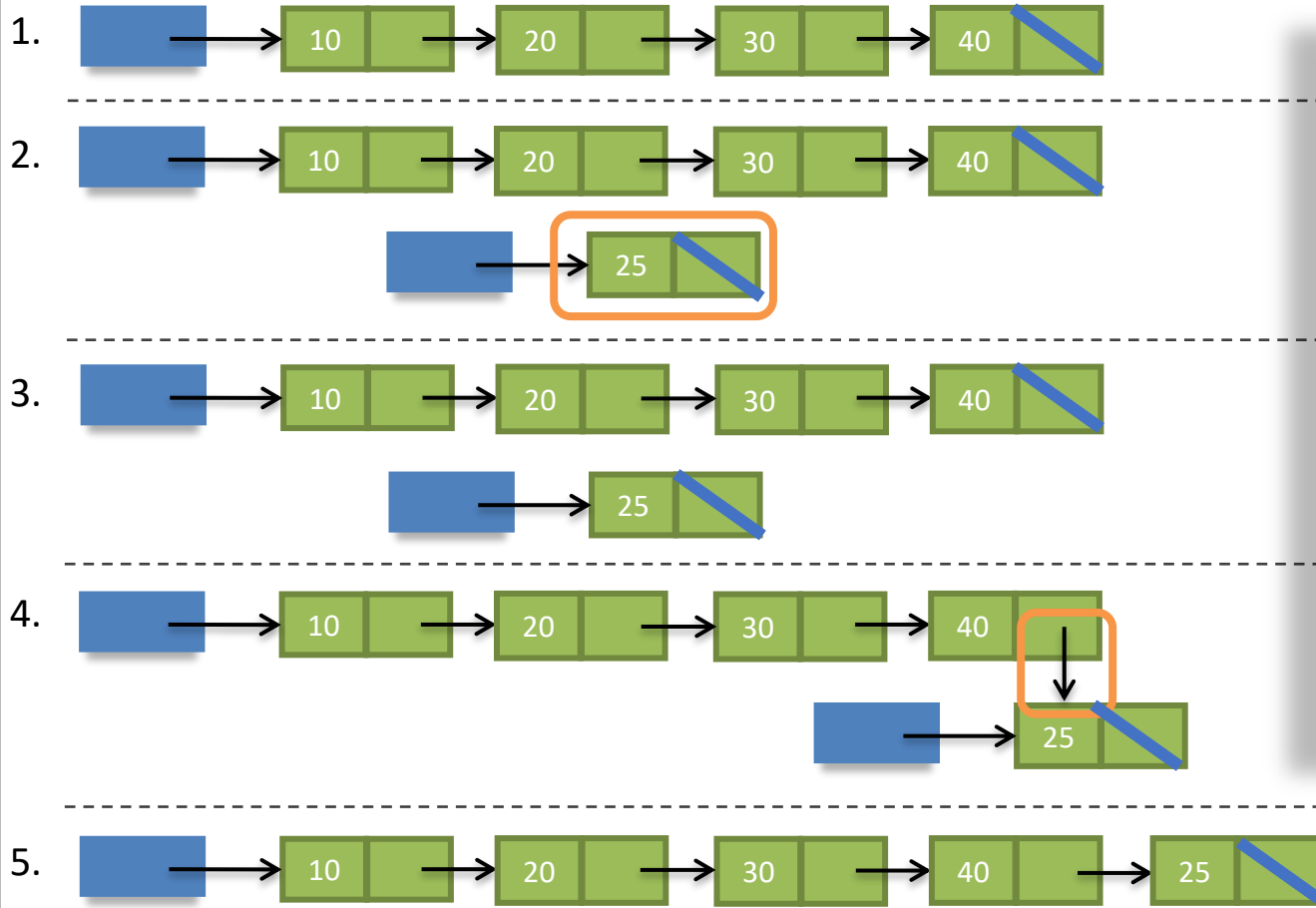


Insert a node in the back



```
1 def insert_at_back(self, data):  
2  
3     new_node = Node(data)  
4  
5     if not self.head:  
6         self.head = new_node  
7         return  
8  
9     last_node = self.head  
10  
11     while last_node.next:  
12         last_node = last_node.next  
13  
14     last_node.next = new_node
```

Insert a node in the back



```
1 def insert_at_back(self, data):  
2  
3     new_node = Node(data)  
4  
5     if not self.head:  
6         self.head = new_node  
7         return  
8  
9     last_node = self.head  
10  
11     while last_node.next:  
12         last_node = last_node.next  
13  
14     last_node.next = new_node
```

Insert a node at the front

- What is common issue to both cases?

- Empty list

- 



- Inserting a node at index 0



Need to modify the content
of head pointer

Insert a node at the front

- What is common issue to both cases?
 - Empty list



Insert a node at the front

- What is common issue to both cases?
 - Empty list



```
def insert_at_front(self, data):  
    # Step 1: Create a new node  
    new_node = Node(data)
```

Insert a node at the front

- What is common issue to both cases?

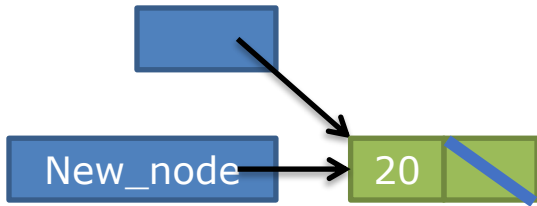
- Empty list



```
def insert_at_front(self, data):  
    # Step 1: Create a new node  
    new_node = Node(data)  
    # Step 2: Point new node to the current head  
    new_node.next = self.head #None
```

Insert a node at the front

- What is common issue to both cases?
 - Empty list



Need to modify the content
of head pointer

```
def insert_at_front(self, data):  
    # Step 1: Create a new node  
    new_node = Node(data)  
    # Step 2: Point new node to the current head  
    new_node.next = self.head  
    # Step 3: Update the head to point to the new node  
    self.head = new_node
```


Insert a node at the front

- What is common issue to both cases?

- Empty list

-  → None

- Inserting a node at index 0



Insert a node at the front

- What is common issue to both cases?

- Empty list



- Inserting a node at index 0



```
def insert_at_front(self, data):  
    # Step 1: Create a new node  
    new_node = Node(data)
```

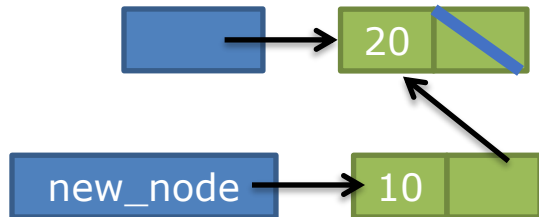
Insert a node at the front

- What is common issue to both cases?

- Empty list



- Inserting a node at index 0



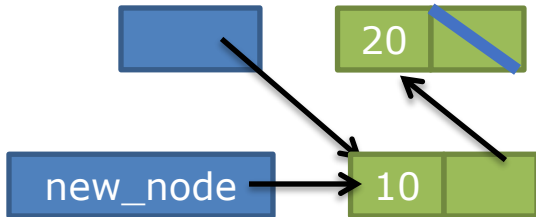
```
def insert_at_front(self, data):  
    # Step 1: Create a new node  
    new_node = Node(data)  
    # Step 2: Point new node to the current head  
    new_node.next = self.head
```

Insert a node at the front

- What is common issue to both cases?
 - Empty list



- Inserting a node at index 0



```
def insert_at_front(self, data):  
    # Step 1: Create a new node  
    new_node = Node(data)  
    # Step 2: Point new node to the current head  
    new_node.next = self.head  
    # Step 3: Update the head to point to the new node  
    self.head = new_node
```

Insert a node at the front

- What is common issue to both cases?

- **Empty list**



- Inserting a node at index 0



**Need to modify the content
of head pointer**

```
def insert_at_front(self, data):  
    # Step 1: Create a new node  
    new_node = Node(data)  
    # Step 2: Point new node to the current head  
    new_node.next = self.head  
    # Step 3: Update the head to point to the new node  
    self.head = new_node
```

Insert a node at the front

- What is common issue to both cases?

- **Empty list**



- Inserting a node at index 0



```
def insert_at_front(self, data):  
    # Step 1: Create a new node  
    new_node = Node(data)  
    # Step 2: Point new node to the current head  
    new_node.next = self.head  
    # Step 3: Update the head to point to the new node  
    self.head = new_node
```

**Need to modify the content
of head pointer**

```
1 def insert_at_back(self, data):  
2  
3     new_node = Node(data)  
4  
5     if not self.head:  
6         self.head = new_node  
7         return  
8  
9     last_node = self.head  
10  
11     while last_node.next:  
12         last_node = last_node.next  
13  
14     last_node.next = new_node
```

Append node at given index

```
1  class Node:
2      def __init__(self, data):
3          self.data = data
4          self.next = None
5
6  class LinkedList:
7      def __init__(self):
8          self.head = None
9
10
11     def insert_at_back(self, data):
12         new_node = Node(data)
13         if not self.head:
14             self.head = new_node
15             return
16         last_node = self.head
17         while last_node.next:
18             last_node = last_node.next
19         last_node.next = new_node
20
21
22
23     def insert_at_front(self, data):
24         new_node = Node(data)
25         new_node.next = self.head
26         self.head = new_node
27
28     def display(self):
29         current = self.head
30         while current:
31             print(current.data, end=" -> ")
32             current = current.next
33         print("None")
34
35     def findAt(self, index):
36         current = self.head
37         if not current:
38             return None
39         while index>0:
40             current = current.next
41             if not current:
42                 return None
43             index-=1
44         return current
```

Insert node at given index

```
1 def insert(self, data, index):
2     # Create a new node with the given data
3     new_node = Node(data)
4
5
6     # If list is empty or inserting at head
7     if self.head is None or index == 0:
8         new_node.next = self.head
9         self.head = new_node
10        return True
11
12
13    # Start at the head of the list
14    current = self.head
15    count = 0
```

```
19 # Traverse until index-1 position
20 # (node before where we want to insert)
21 while current and position < index - 1:
22     current = current.next
23     count += 1
24
25 # If current is None, index was too large
26 if not current:
27     print("Index out of range")
28     return False
29
30 # Insert the new node by updating pointers:
31 # 1. New node points to current's next node
32 # 2. Current node points to new node
33 new_node.next = current.next
34 current.next = new_node
35 return True
36
```

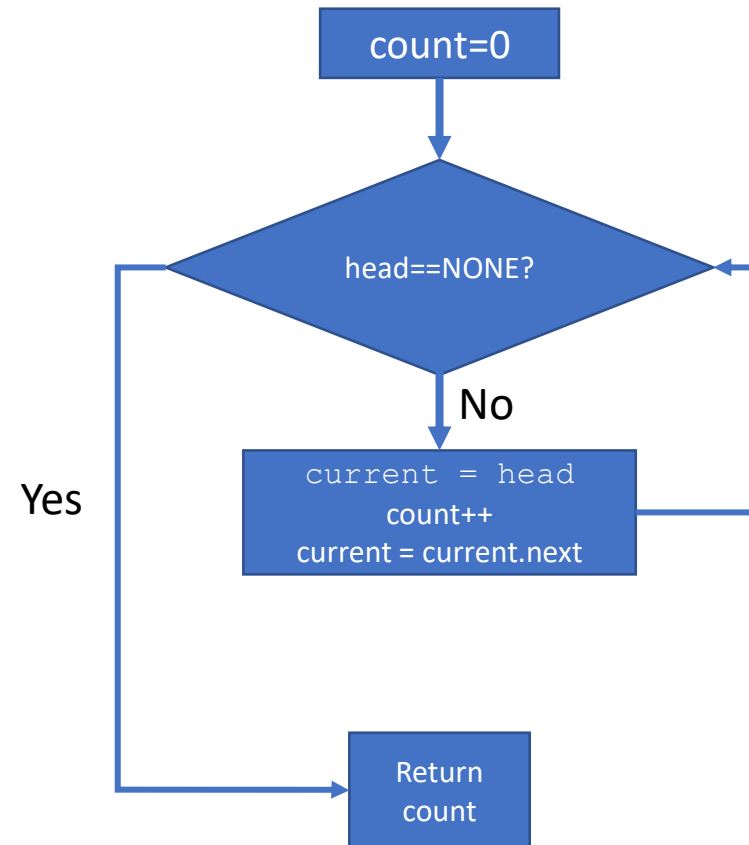

Size of The LinkedList (not smart solution)

Given

- the head reference of the linked list

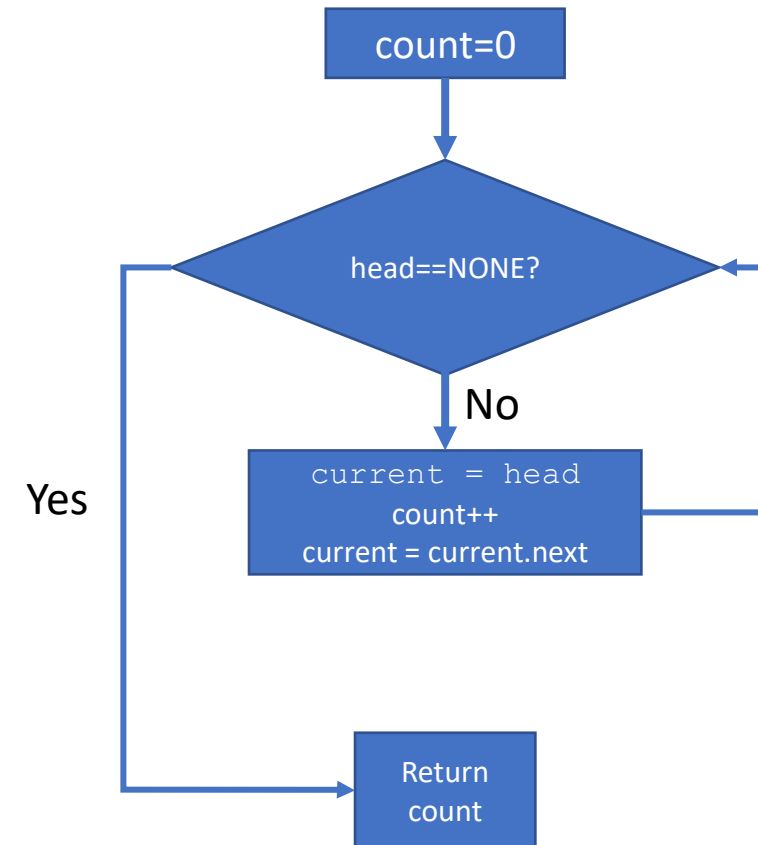
Return the number of nodes in the linked list

1. Declare a counter and initialize it to zero
2. Check the reference whether is None or not
3. Increase the counter
4. Current move to next node
5. Repeat step 2
6. Return the counter



Size of The LinkedList (not smart solution)

```
1 def sizeList(head):  
2  
3     # Initialize counter  
4     count = 0  
5  
6     # Start at head  
7     current = head  
8  
9     # Traverse the list  
10    while current is not None:  
11        count += 1 # Increase counter  
12        current = current.next # Move to next node  
13  
14    return count # Return final count
```



The Linked List

- Just introduce a new member in the linked list class, *size*
- Initialize *size* as zero
- When you add or remove a node, increase or decrease *size* by one accordingly

```
1 class ListNode:
2     def __init__(self, item):
3         self.item = item
4         self.next = None
5
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9         self.size = 0
```

Size of The Linked List (better solution)

- Just introduce a new member in the linked list class, *size*
- Initialize *size* as zero
- When you add or remove a node, increase or decrease *size* by one accordingly

```
1 class ListNode:
2     def __init__(self, item):
3         self.item = item
4         self.next = None
5
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9         self.size = 0
10
11 def sizeList(ll): # ll = LinkedList() #
12     return ll.size
13
14 ll = LinkedList() # Example usage #
15 print(sizeList(ll))
```

findNode

```
1 def findNode(head, index):
2
3     if head is None or index < 0:
4         return None
5
6     cur = head
7     while index > 0:
8         cur = cur.next
9         if cur is None:
10             return None
11         index -= 1
12     return cur
```

```
1 def findNode2(ll, index):
2     # Check if list is empty or index is invalid
3     if ll.head is None or index < 0 or index >= ll.size:
4         return None
5
6     # Start traversing from head
7     cur = ll.head
8     while index > 0:
9         cur = cur.next
10        index -= 1
11    return cur
```

insertNode

```
1 def insertNode(head, index, item):
2     newNode = ListNode(item)
3
4     # Scenario 1: Inserting into empty list
5     if head is None:
6         if index == 0:
7             head = newNode
8             return True
9         return False
10
11     # Scenario 2: Inserting at beginning
12     if index == 0:
13         newNode.next = head
14         head = newNode
15         return True
16
17     # Scenario 3: Inserting in middle or end of list
18     prev = findNode(head, index - 1)
19     if prev is not None:
20         newNode.next = prev.next
21         prev.next = newNode
22         return True
23     return False
```

```
1 def insertNode2(ll, index, item):
2     newNode = ListNode(item)
3
4     # Case 1: Inserting at the beginning
5     if index == 0:
6         newNode.next = ll.head
7         ll.head = newNode
8         ll.size += 1
9         return True
10
11     # Case 2: Inserting anywhere else
12     pre = findNode2(ll, index - 1)
13
14     if pre is not None:
15         newNode.next = pre.next
16         pre.next = newNode
17         ll.size += 1
18         return True
19
20     return False
```

Remove a Node (Lab Questions)

- Remove a node at

1. Front



2. Middle



3. Back



Real world Application – Task Scheduling

Task Scheduling :

Linked lists are often used in real-world task scheduling, especially in operating systems and other environments where efficient management of tasks or processes is critical.

Advantages of Linked Lists in Task Scheduling

- Efficient Insertion/Deletion: Linked lists allow for quick insertion and deletion of tasks, which is essential in environments where tasks frequently enter and exit the queue.
- Dynamic Sizing: No pre-allocation of memory is needed, making linked lists ideal for systems with fluctuating task loads.

Summary

- Referential Arrays
- Customized Data Types
- Singly Linked List
- Abstract Data Type (ADT)
 - Display data (Print all data)
 - Search a node
 - Add a node
 - Remove a node
 - Get the size of the linked list